

第七周周报

第 2 组 OS 课设周报-第 7 周-杜昊阳-富思涵-赫金科-康忆文-卢彦芸-陆昱衡-米梓润-吴安然

本周开始周报主要描述上周计划的执行情况及下周工作计划等内容

一、本周计划执行情况

1. 内存管理：杜昊阳，赫金科

- 段页式内存管理方式已实现

内核启动时根据 `MMU_MODE` 配置启用段页式内存管理，完成物理内存的恒等映射 (identity mapping) 并开启分页机制。

- `memmap` 程序增强

用户态程序 `apps/memmap/memmap.c` 增加对 MMU 模式的显示，通过 `api_getmmumode` 系统调用获取当前内存管理方式。

- 内核初始化流程

`kernel/bootpack.c:110` 调用 `paging_identity_map_init`，若失败则系统挂起。

- 分页初始化函数

`kernel/memory.c:paging_identity_map_init` 完成：

- a. 分配页目录 (PD) 和所需页表 (PT) 内存。
- b. 对 `0 ~ memtotal` 物理地址建立恒等映射 (页表项 = 物理地址 | `0x007`)。
- c. 对 VRAM 区域特殊处理，确保显存映射正确。
- d. 将页目录地址存入 `kernel_cr3`，执行 `store_cr3` 加载页表，设置 `CR0.PG` 位开启分页。

- 内核数据结构

- `kernel/memory.c` 中定义 `kernel_cr3` 全局变量，保存当前页目录物理地址。
- `kernel/bootpack.h` 中定义 `MMU_MODE_SEG_PAGE` 宏 (值为 1)，用于编译时选择段页模式。
- `CR0_PAGING_ENABLE` 宏 (`0x80000000`) 用于开启分页。

- 系统调用链路

- 用户态封装: `apps/apilib/api042.nas` 实现 `api_getmmumode`, 通过 `INT 0x40` 调用内核。
- 内核分发: `kernel/console.c` 中 `case 42` 对应 `cmd_getmmumode`, 返回当前 MMU 模式。
- 用户态程序
`apps/memmap/memmap.c` 调用 `api_getmmumode`, 将返回的模式 ID 转换为字符串 ("SEGMENT" 或 "SEG_PAGE") 并显示在窗口上。

2. 中断: 吴安然

完成扩展实验 (链表插入复杂度测量、`MAX_TIMER` 扩容验证) 及全周期总结, 形成完整实验报告。

- 扩展实验设计

新增 TC-10: 通过在不同背景定时器数量 ($n=0, 50, 100, 200$) 下测量插入耗时, 验证 `timer_settime` 链表插入的 $O(n)$ 特性。新增 TC-11: 将 `MAX_TIMER` 从 500 扩容至 800, 验证编译运行正常且可用槽位相应增加。

- 全周期工作总结

梳理第四至第七周全部工作: 第四周完成代码阅读与原理归纳; 第五周实现 TC-01~TC-05; 第六周完成 TC-06~TC-09 协同验证; 本周完成 TC-10~TC-11 及总结汇报。共完成 11 个测试用例, 覆盖正确性、压力、边界、调度协同、中断计数、批量取消、池恢复、复杂度、扩容九个维度, 全部通过。

- 定时器管理机制评价

总结了静态池+有序链表设计的优点 (简洁、早期返回优化 $O(1)$ 、无内存碎片) 与局限 (插入 $O(n)$ 、分配线性扫描、`count` 溢出回绕)。

- 问题修复记录

记录了三项发现的问题: TC-01 断言指针比较误判、`cmdline` 缓冲区溢出、QEMU 键盘输入干扰, 均已修复。

- 本周产出

- a. 完整实验报告 (含 TC-01~TC-11 测试结果表)。
- b. 扩展实验 TC-10/TC-11 代码与测试输出。
- c. 全周期工作总结文档。

3. 进程调度: 富思涵, 卢彦芸, 康忆文

从用户态启动内核态竞争条件、解决方案、同步场景 (系统调用练习)

- 用户态触发入口

用户程序 `syncdemo.c` 调用 `api_syncdemo_start`, 该 API 在 `api060.nas` 里通过 `INT 0x40`、`EDX=60` 进入内核。

- 内核侧系统调用分发

系统调用总入口在 `console.c:627` 的 `hrb_api`, 其中 `case 60` 会打开同步监视窗口并触发演示逻辑 (`open_syncmon`)。

- 内核态“竞争条件 + 解决方案 + 同步场景”

无锁竞争任务: `task_race_unsafe`, 故意做非原子读改写, 展示 `lost update`。

有锁竞争任务: `task_race_safe`, 用内核信号量保护临界区。

同步场景: 读者-写者任务 (`task_rw_reader` / `task_rw_writer`)。

启动入口: `syncdemo_start_once` 与 `open_syncmon` (首次打开时启动任务集, 后续只开监视窗口)。

- 可视化监视

监视窗口周期刷新逻辑在同文件的 `task_syncmon/sync_mon_refresh`, 直接显示 `unsafe lost`、`safe lost`、读写违规计数等指标, 方便实验汇报。

用户态通过系统调用产生竞争条件, 给出解决方案, 并实现生产者消费者

- 用户态“竞争条件”程序已实现

无锁版本代码在 `race.c`, 逻辑为 `api_get_shared` → 加 1 → `api_set_shared`, 多进程并发时会出现结果偏小 (竞态)。

- 用户态“解决方案 (信号量保护)”

加锁版本在 `safe.c`; 逻辑为 `api_sem_wait` 后再读改写共享变量, 结束后 `api_sem_post`。对比 `race` 与 `safe` 可以直接证明同步机制有效。

- 共享变量与信号量系统调用链路

用户态封装: `api061.nas` 到 `api065.nas`

内核分发: `console.c:627` 中 `case 61~65` 对应 `get/set shared`、`sem wait/post`、`reset`。

内核数据结构与实现: `bootpack.c` 中 `g_user_shared_var`、`g_user_sem`、`ksem_wait/ksem_signal`。

- 生产者消费者场景

- 用户态程序:

生产者 `producer.c`

消费者 consumer.c

重置程序 syncrst.c

- 系统调用封装：

api066.nas 到 api068.nas

- 内核实现：

bootpack.c 中环形缓冲区 g_pc_buffer + 三信号量（互斥、空槽、满槽），是标准生产者消费者模型。

4. 文件系统：米梓润，陆昱衡

- 更新 memf 文件创建的返回内容，使之更明确
- 设计并实现一个可演示方案：在用户程序中选取不同类型的变量（全局/局部/静态/堆），展示它们从逻辑地址到物理地址的转换过程，并进行验证。
- 完善部分文件操作指令

二、下周工作计划

1. 内存管理：杜昊阳，赫金科

- 高地址空间映射实现
- 内核虚拟地址定义
- 设计高地址映射逻辑

2. 中断：吴安然

- 根据答辩反馈微调实验报告。
- 将代码改动合并至团队主分支，提交最终版本。
- 整理测试日志与截图，打包提交至课程指定位置。
- 协助组内其他同学复现测试用例。

3. 进程调度：富思涵，卢彦芸，康忆文

- 针对上周实现的用户态程序通过系统调用进行同步产生竞争条件以及解决方案进行深入优化，进一步探究本任务的核心，并思考是否有更简便的实现方案
- 理解丰富上周实现的生产者消费者同步场景，设计更复杂的场景模式用于展示

- 在老师提供的基础版本 `haribote` 的基础上，研究任务从内核态跳转到用户态执行，并通过系统调用进入内核态，系统调用结束又返回内核态的过程。
- 根据现有代码，形成初步项目文档，记录实现方法以及每一个修改的代码具体功能

4. 文件系统：米梓润，陆昱衡

- 继续更新 `memf` 返回内容，使之更人性化，降低学习难度
- 重新规划 `api` 分配
- 继续研究 `LDT` 内容
- 验证可演示方案，增强鲁棒性